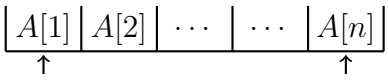
1) **Cálculo Linear**: O algoritmo abaixo percorre um vetor A de n elementos para somá-los. Preencha a tabela de custos:



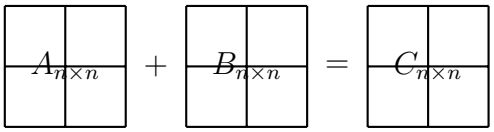
Percorre as posições sequencialmente

SOMA-VETOR(A, n)

	instrução	custo	vezes
1	$soma = 0$	_____	_____
2	**para** $i = 1$ **até** n:	_____	_____
3	$\quad soma = soma + A[i]$	_____	_____
4	**retornar** $soma$	_____	_____

Resposta:

2) **Cálculo Quadrático**: Duas matrizes A e B de ordem $n \times n$ são somadas. Analise o impacto dos laços aninhados:



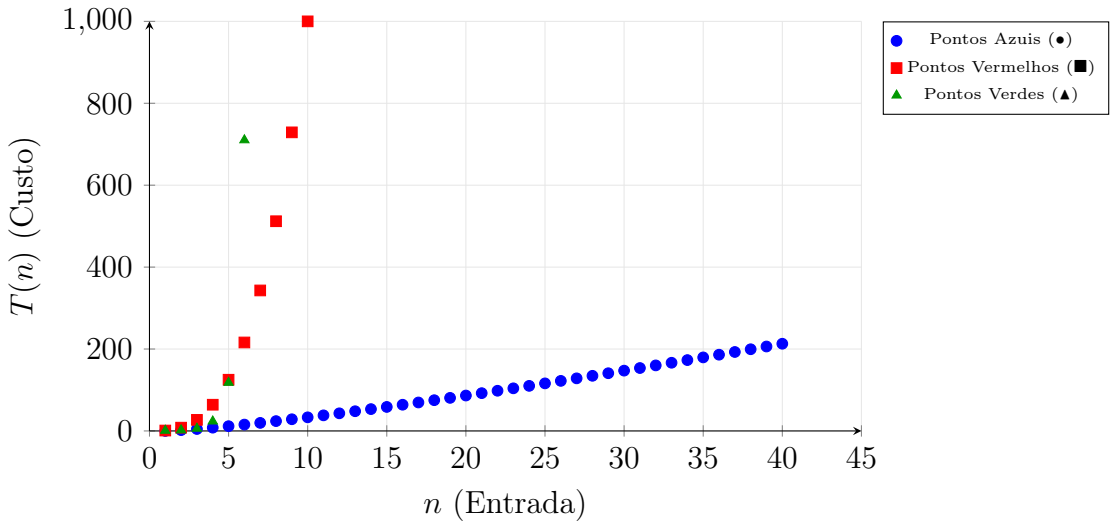
Cada célula (i, j) exige um cálculo

SOMA-MATRIZ(A, B, n)

	instrução	custo	vezes
1	**para** $i = 1$ **até** n:	_____	_____
2	$\quad$ **para** $j = 1$ **até** n:	_____	_____
3	$\quad\quad C[i, j] = A[i, j] + B[i, j]$	_____	_____
4	**retornar** C	_____	_____

Resposta:

3) **Análise Gráfica de Pontos**: Com base no comportamento das curvas no gráfico abaixo, assinale a alternativa que associa corretamente a cor/formato do ponto à sua respectiva complexidade:



a) Azul: $O(n^3)$; Vermelho: $O(n!)$; Verde: $O(n \log n)$.

b) Azul: $O(n!)$; Vermelho: $O(n^3)$; Verde: $O(n \log n)$.

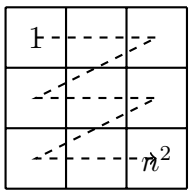
c) Azul: $O(n \log n)$; Vermelho: $O(n^3)$; Verde: $O(n!)$.

d) Azul: $O(n \log n)$; Vermelho: $O(n!)$; Verde: $O(n^3)$.

4) **Busca em Matriz (Busca Exaustiva)**: Considere uma matriz quadrada M de ordem n, que armazena um total de n^2 elementos. O algoritmo abaixo realiza uma varredura completa, percorrendo cada linha e cada coluna para localizar um valor específico chamado 'alvo'.

   No cenário de **pior caso**, o elemento buscado está na última célula da última linha (posição $M[n, n]$) ou não está presente na matriz, o que exige que o software verifique absolutamente todas as posições.

   - **Exemplo 1**: Em uma matriz 3×3 ($n = 3$), o algoritmo verificará até 9 elementos (3^2).
   - **Exemplo 2**: Em uma matriz 100×100 ($n = 100$), o algoritmo verificará até 10.000 elementos (100^2).



Varredura completa

BUSCA-MATRIZ$(M, n, alvo)$

	instrução	custo	vezes
1	**para** $i = 1$ **até** n:	_____	_____
2	**para** $j = 1$ **até** n:	_____	_____
3	**se** $M[i, j] == alvo$:	_____	_____
4	**retornar** (i, j)	_____	_____
5	**retornar** NULO	_____	_____

Resposta:

5) **Laços Independentes vs Aninhados**: O algoritmo abaixo processa o vetor A de duas formas diferentes. Qual termo domina o custo final?

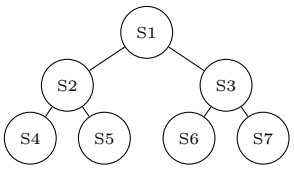
ALGORITMO-MISTO(A, n)

	instrução	custo	vezes
1	**para** $i = 1$ **até** n:	_____	_____
2	print$(A[i])$	_____	_____
3	**para** $j = 1$ **até** n:	_____	_____
4	**para** $k = 1$ **até** n:	_____	_____
5	print$(A[j] + A[k])$	_____	_____

Resposta:

6) **Infeção de Rede (Ransomware)**: Analise o cenário de um ataque cibernético onde um **Ransomware** altamente agressivo utiliza uma técnica de replicação recursiva. Ao infectar um servidor, o código malicioso busca vulnerabilidades em outros dois servidores conectados à mesma rede e inicia o processo de infecção neles simultaneamente. Este comportamento se repete para cada novo servidor comprometido, avançando por diversos níveis de profundidade na infraestrutura de TI.

Qual a complexidade Big-O desta propagação em relação ao **nível** de profundidade da rede?



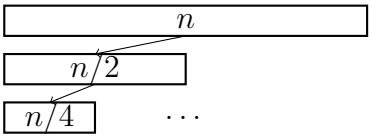
```
INFECTA-REDE(nível)
1   se nível > 0:
2      Infectar Servidor A (INFECTA-REDE(nível −1))
3      Infectar Servidor B (INFECTA-REDE(nível −1))
4      Criptografar arquivos do servidor atual
```

Resposta:

7) **Simplificação Assintótica**: Dada a função de custo $T(n) = 5n^2 + 100n + 500$, identifique a complexidade Big-O.

Resposta:

8) **Busca Binária**: Em um vetor de $n = 1.000.000$ elementos ordenados, a busca binária divide o problema sucessivamente por 2.

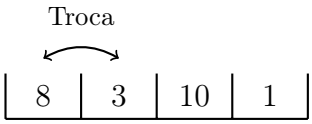



```
BUSCA-BINARIA(A, n, alvo)
1   início = 1, fim = n
2   enquanto início ≤ fim:
3      meio = (início + fim)/2
4      se A[meio] == alvo: retornar meio
5      senao se A[meio] < alvo: início = meio + 1
6      senao: fim = meio − 1
7   retornar NULO
```

Qual a complexidade O e o número máximo de comparações para $n = 1.000.000$?

Resposta:

9) O algoritmo abaixo compara pares adjacentes e os troca. Preencha a tabela de análise:



ORDENA(A, n)

	instrução	**custo**	**vezes**
1	**para** $i = 1$ **até** n:	_____	_____
2	**para** $j = 1$ **até** $n - 1$:	_____	_____
3	**se** $A[j] > A[j + 1]$:	_____	_____
4	Troca$(A[j], A[j + 1])$	_____	_____

Resposta:

10) **Comparação de Curvas**: O algoritmo A possui $100n$ e o algoritmo B possui n^2. Determine a partir de qual valor de n o Algoritmo A torna-se mais eficiente.

Resposta:

11) **Questão de Concurso (CESPE / CEBRASPE - 2025 - Polícia Federal)**: No que se refere ao SonarQube, às estruturas de dados e à complexidade de algoritmos, julgue o item subsecutivo:

"Para grandes volumes de dados, um algoritmo com complexidade de tempo $O(n)$ (linear) é considerado menos eficiente que um algoritmo com complexidade de tempo $O(n \log n)$, uma vez que o crescimento linear é mais acentuado que o crescimento logarítmico."

( ) Certo

( ) Errado

12) **Questão de Concurso (IV - UFG - 2025 - Câmara de Morrinhos - GO - Analista de TI)**:
Considere o seguinte código escrito em Python 3:

```
def busca_binaria(lista, elemento):
    inicio = 0
    fim = len(lista) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2
        if lista[meio] == elemento:
            return meio
        elif lista[meio] < elemento:
            inicio = meio + 1
        else:
            fim = meio - 1

    return -1
```

A complexidade de tempo desse algoritmo em termos da notação Big-O é:

A) $O(1)$

B) $O(n)$

C) $O(n^2)$

D) $O(\log n)$

E) $O(n!)$

13) **Questão de Concurso (COPEVE-UFAL - 2023 - UFAL - Analista de TI)**:
Depois de pensar sobre determinado problema, João fez um rascunho de uma função, produzindo o algoritmo em pseudocódigo do quadro:

```
função calculo_A(n)
    k = 0
    se (n > 1000) então
        para i de 1 até n faça
            para j de 1 até n faça
                k = (k * k) + j
            fim para
        fim para
    senão
        para j de 1 até n faça
            k = (k * j) + 2
        fim para
    fim se
    retorne k
```

Qual a complexidade de tempo dessa função na notação Big-O?

A) $O(n)$

B) $O(n^2)$

C) $O(n^3)$

D) $O(\log n)$

E) $O(n \log n)$